CS 5180 – Assignment #2

Maximum Points: 100 pts.

Bronco ID: |__|__|__|__|__|__|__|__|__|

Last Name: _____

First Name: _____

Note 1: Your submission header must have the format as shown in the above-enclosed rounded rectangle.
Note 2: Homework is to be **done individually**. You may discuss the homework problems with your fellow students, but you are NOT allowed to copy – either in part or in whole – anyone else's answers.
Note 3: Your deliverable should be a **.pdf file** submitted through **Gradescope** by the deadline. Do not forget to **assign a page to each of your answers** when making a submission. In addition, source code (.py files) should be added to an **online repository (e.g., GitHub)** to be downloaded and executed later.
Note 4: All submitted materials must be legible. Figures/diagrams must be of good quality.
Note 5: Please use and check the Canvas discussion for further instructions, questions, answers, and hints. The bold words/sentences provide information for a complete or accurate answer.

1. [16 points]. Given the term-frequency inverted index below, answer the following questions.

"and": [1: 1, 2: 1, 3: 2]	"over": [1: 1]
"brown": [1: 1, 2: 2]	"quick": [1: 1]
"dog": [1: 2, 2: 2, 3: 2]	"ran": [3: 1]
"forest": [3: 1]	"riverbank": [2: 1]
"fox": [1: 1, 2: 1, 3: 1]	"sat": [2: 1]
"jumped": [1: 1]	"slow": [2: 1]
"lazy": [1: 1, 2: 1, 3: 1]	"the": [1: 3, 2: 4, 3: 4]
"met": [3: 1]	"through": [3: 1]
"near": [2: 1]	"with": [2: 1]

   a. [4 points]. What is the size of the vocabulary?
   b. [4 points]. What is the total number of tokens in the collection?
   c. [4 points]. Which document is the longest?
   d. [4 points]. What is the term with the second highest collection frequency?

2. [16 points]. Use the positional inverted index below to answer the following questions.

"and": {1: [5], 2: [5], 3: [3, 10]}	"over": {1: [10]}
"brown": {1: [3], 2: [3, 15]}	"quick": {1: [2]}
"dog": {1: [8, 12], 2: [4, 16], 3: [5, 14]}	"ran": {3: [6]}
"forest": {3: [9]}	"riverbank": {2: [12]}
"fox": {1: [4], 2: [8], 3: [2]}	"sat": {2: [9]}
"jumped": {1: [9]}	"slow": {2: [2]}
"lazy": {1: [7], 2: [7], 3: [13]}	"the": {1: [1, 6, 11], 2: [1, 6, 11, 14], 3: [1, 4, 8, 12]}
"met": {3: [11]}	"through": {3: [7]}
"near": {2: [10]}	"with": {2: [13]}

   a. [4 points]. Which document(s) contain the exact phrase "the lazy dog"?
   b. [4 points]. Which document(s) contain "fox" and "dog" within 3 positions of each other?
   c. [4 points]. Suppose we add the following extent list to the positional inverted index. `title:{1:[3,6),2:[6,9),3:[4,7)}`. Which document(s) contain the term "fox" in the title?
   d. [4 points]. Construct the document-term matrix using term-frequency sorting terms alphabetically.

3. [16 points]. You are given the following POS tagged documents:

The/DT grand/JJ jury/NN commented/VB on/IN a/DT number/NN of/IN other/JJ topics/NN.
A/DT skilled/JJ attacker/NN can/MD exploit/VB a/DT misconfigured/JJ web/NN server/NN easily/RB.

Allowed POS Tags:

DT	Determiner
JJ	Adjective
NN	Noun (all numbers)
VB	Verb (all tenses)
MD	Modal
RB	Adverb
IN	Preposition

Chunking Rules

NP → DT? JJ* NN+
NP → NN IN
VP → MD VB
VP → VB RB?
VP → VB IN

a. [4 points]. Identify all noun phrases in the documents using the chunking rules provided*.
b. [4 points]. From the noun phrases in (a), list only those containing exactly two tokens.
c. [4 points]. Identify all verb phrases in the documents using the chunking rules provided*.
d. [4 points]. From the verb phrases in (c), list only those containing exactly two tokens.

* Apply the chunking rules from left to right. When multiple rules match starting at the same token, choose the longest chunk of tokens. Once a chunk is selected, continue scanning from the next token after that chunk to avoid overlaps. Show all your derivations.

4. [16 points]. Write a RegEx that matches ONLY the indicated strings. Requirements: do not hardcode by explicitly listing all alternatives (e.g., a|b|c|).

a. [2 points]. ac, abc, abbc, abbbc, abbbbc, ...
b. [2 points]. A2, B4, E9, D0, E2, P3, ..., Z1
c. [2 points]. 016, 14, 29, 332, 428, 535, 13, ..., 97
d. [2 points]. aa, ba, ca, da, ea, fa…, za
e. [2 points]. a, aba, ababa, abababa, ababababa, ...
f. [2 points]. a0b, a1b, a2b, a3b, ..., a9b
g. [2 points]. 0b, 0a, 0ba, 1b, 1a, 1ba, …, 9b, 9a, 9ba
h. [2 points]. 00, 011, 0111, 01111, ...

5. [16 points]. Consider the following two postings lists augmented with skip pointers:

A: [4, 14, 18, 31, 41→46, 44, 46, 49]
B: [3, 10→14, 12, 14, 15, 23, 40, 46]

Using the algorithm INTERSECTWITHSKIPS, enumerate each comparison step performed during the intersection of A and B. At each step, indicate the action taken: move A, move B, skip A, skip B, match. Write the final result of the intersection.

6. [20 points]. Complete the Python program (SPIMI_index.py) to implement a simplified SPIMI-based inverted index construction pipeline following the strict specifications below.

1. Dataset:
    a. File: corpus.tsv.
    b. Format: doc_id<TAB>text.
    c. Encoding: UTF-8.
2. Text Processing:
    a. Use CountVectorizer for tokenization and enable English stopword removal.

3. Index Type:
    a. You must build a binary inverted index. Format: term → sorted list of docIDs
    b. Postings lists must be strictly increasing (sorted ascending).
4. Block Construction (SPIMI Phase):
    a. Divide the 1,000 documents into 10 blocks of 100 documents each, in ascending docID order:
        i. Block 1: docIDs 1–100
        ii. Block 2: docIDs 101–200
        iii. …
        iv. Block 10: docIDs 901–1000
    b. For each block:
        i. Build an in-memory inverted index using the SPIMI algorithm.
        ii. Sort terms lexicographically before writing the block to disk.
        iii. Write each block to disk as:

```
block_1.txt, block_2.txt, ..., block_10.txt
```

5. Block File Format:
    a. Each block file must:
        i. Be UTF-8 encoded.
        ii. Contain one term per line.
        iii. Use the following format (no spaces): `term:docID1,docID2,docID3,...`
        iv. Example:

```
analysis:3,17,402
model:2,9,100
system:1,5,77,212
```

6. Final Merge Phase
    a. After all 10 block files are written, you must merge them into a final inverted index:

```
final_index.txt
```

    b. Merge Requirements
        i. Perform a multiway merge over the 10 block files.
        ii. During merging, you may keep at most 100 term-lines buffered per input block file (i.e., you may not load entire block files fully into memory.)
        iii. During merging, you may buffer at most 500 output merged term-lines in memory before flushing them to disk.
        iv. Merge must preserve:
            1. lexicographic term order
            2. sorted postings
            3. deduplicated postings
        v. Final output format must be identical to block format.

Important Note: Answers to all questions should be written clearly, concisely, and unmistakably delineated. You may resubmit multiple times until the deadline (the last submission will be considered).

NO LATE ASSIGNMENTS WILL BE ACCEPTED. ALWAYS SUBMIT WHATEVER YOU HAVE COMPLETED FOR PARTIAL CREDIT BEFORE THE DEADLINE!